CSA 期中考复习笔记

UNITS 1-4

根据《CSA WorkBook-1》前四个 units 的题型与考点整理

使用方式

第一遍按 Unit 阅读；第二遍只看“高频陷阱”和代码模板；考前最后 20 分钟使用末尾 Quick Sheet。所有关键名词与代码保留 English，其余说明使用中文。

覆盖范围

- Unit 1: Primitive Types
- Unit 2: Using Objects
- Unit 3: Boolean Expressions and if Statements
- Unit 4: Iteration

基于题目归纳，不复刻原题；示例代码为重新编写。

总览：这四个 Units 在考什么

Unit	核心能力	题目最爱考的失误
1 Primitive Types	追踪 statement execution；计算 expression；选择 data type；casting	integer division、operator precedence、cast 太晚、负数 rounding
2 Using Objects	调用 method；处理 String；使用 Math；匹配 parameter/return type	substring 边界、String concatenation、random range、void misuse
3 Boolean / if	判断条件覆盖范围；改写等价 Boolean expression；比较 object	独立 if 与 else-if、De Morgan、short-circuit、== vs equals
4 Iteration	追踪与改写 while loop / for loop	off-by-one、更新位置、condition 永远为 true

统一解题流程

- 先标注每个 variable 的 type。同样的符号 `/`，对 int 与 double 的意义不同。
- 遇到 tracing，建立小表：before condition -> body -> update -> output。不要心算跳步。
- 遇到区间，先写端点：是否 inclusive？把最小值、最大值、临界值代入。
- 遇到 String，写出 index：第一个是 0，最后一个是 length() - 1。
- 遇到“equivalent”，不要只试普通值；优先测试 boundary、0、负数、相等、null。

> 题型信号
> 题目出现 works as intended 时，通常要找 boundary case；出现 same output 时，要同时比较初值、condition、update 与输出位置；出现 cannot cause a run-time error 时，优先检查 null 与 index。

Unit 1 · Primitive Types

Workbook 题目集中在 output tracing、variable declaration、arithmetic、compound assignment、casting 与 numeric range。

1.1 Primitive type 与 variable

Key term	含义 / 选择原则	Example
int	整数；计数、索引、离散数量	int count = 0;
double	含小数的数值；平均数、距离、概率	double average = 0.0;
boolean	只有 true / false；表示条件或状态	boolean ready = false;
declaration	声明 type 与变量名	int score;
initialization	第一次赋值	int score = 10;
assignment	把右侧计算结果存入左侧	score = score + 2;
constant	用 final 表示之后不可重新赋值	final double PI = 3.14159;

1.2 Arithmetic operators

Operator	规则	容易错的点
+ - *	按 precedence 运算；括号优先	不要按从左到右忽略 *
/	两边都是 int -> integer division；只要一边是 double -> decimal division	7 / 3 是 2；7 / 3.0 是 2.333...
%	remainder；常用于奇偶、末位数字、周期	num % 10 取个位；num / 10 去个位
++ --	增加或减少 1	tracing 时要按执行顺序更新
+= -= *= /= %=	先计算，再存回原 variable	num += num 后值翻倍

```
int num = 75;
int onesDigit = num % 10;  // 5
int tensDigit = num / 10;  // 7

int x = 5;
x += 6;    // 11
x -= 3 / 2; // 10, because 3 / 2 is 1
```

> 高频陷阱
> 赋值不是数学等式。x = x + 1 表示先读取旧 x，再把新值存回 x。交换两个值必须借助 temporary variable。

1.3 Casting：先算还是先转？

casting 只影响它直接作用的 expression。把已经完成的 integer division 再 cast 成 double，丢失的小数不会回来。

```
int sum = 9;
double wrong = (double) (sum / 2); // 4.0
double right = (double) sum / 2;   // 4.5
double alsoRight = sum / 2.0;      // 4.5

double value = 4.9;
int truncated = (int) value;       // 4, not 5
```

Expression	Result	Reason
double a = 9 / 4;	2.0	右侧先做 integer division
double b = 9.0 / 4;	2.25	包含 double
int c = (int) 4.9;	4	cast toward zero
int d = (int) (-4.9);	-4	仍然 toward zero，不是 floor

Rounding

常见正数 rounding 写法是 (int) (x + 0.5)，但对负数不通用。若题目涉及任意正负数，应使用明确的规则或合适的 library method，而不是机械加 0.5。

1.4 Output 与 execution order

- System.out.print 不自动换行；System.out.println 输出后换行。
- String literal 必须用双引号；单个 char 才用单引号，但 AP CSA 常见题主要围绕 String。
- 追踪连续赋值时，每一行都使用上一行已经更新后的值。

```
int a = 4;
int b = 5;
a++;
b++;
int c = a + b;       // 11
System.out.println(a + c); // 16
```

Unit 1 自测清单

- 看到 `/`，能否先判断 operands 的 type？
- 能否用 `% 10` 与 `/ 10` 拆分十进制数字？
- 能否解释 `(double) (sum / 2)` 与 `(double) sum / 2` 的不同？
- 能否逐行追踪 compound assignment 与 increment？
- 能否指出正数 rounding 技巧为什么不适用于负数？

Unit 2 · Using Objects

Workbook 将 String、Math、wrapper class 与 method call 放在同一组考查：核心是读懂 method signature，并严格追踪 return value 与 object state。

2.1 Object、class 与 reference

Key term	含义
class	定义 object 具有什么 data 与 behavior 的蓝图
object / instance	由 class 创建出的具体实体
reference	变量保存的是 object 的引用，不是 object 内容本身
constructor	创建并初始化 object；名字与 class 相同，没有 return type
instantiation	使用 new 创建 object 的过程
null	reference 当前没有指向任何 object；调用 method 会产生 NullPointerException

```
String message = new String("CSA");
// String is the class; message is a reference; new creates an object.
```

2.2 Method signature 与调用

```
public double myMethod(int amount, boolean active) {
    // implementation
}

double result = myMethod(2, false); // valid
```

- 调用时 argument 的数量、顺序、type 必须匹配 parameter。
- void method 不产生可赋值的 value，不能放进 print 或赋值 expression。
- non-void method 的 return type 决定结果能存进什么 variable。int 可以自动扩大为 double，但 double 不能直接缩小为 int。
- 同名 method 只要 parameter list 不同，可以 overload；仅改变 parameter 名字不算不同。

> **高频陷阱**
> 判断 method call 是否能 compile：先看 parameter list，再看 return type，最后看调用位置。不要被 method 名字或 variable 名字干扰。

2.3 String：immutable，但 reference 可重新指向

String 是 object，并且 immutable。method 通常返回一个新 String；原 String 不会被原地修改，除非把结果重新 assignment。

Method / operation	规则	Example result
length()	字符个数；没有 argument	"CompSci".length() -> 7
substring(a, b)	包含 index a，不包含 index b	"ABCDEF".substring(1, 3) -> "BC"
substring(a)	从 index a 到结尾	"ABCDEF".substring(4) -> "EF"
indexOf(s)	第一次出现的 index；找不到返回 -1	"banana".indexOf("na") -> 2
equals(s)	比较 String 内容	"AP".equals("AP") -> true
compareTo(s)	字典顺序：负 / 0 / 正；不要假定只返回 -1/0/1	"A".compareTo("B") < 0
+	只要链中已进入 String concatenation，之后继续拼接	"x" + 1 + 2 -> "x12"

```
String oldText = "ABCDEF";
String newText = oldText.substring(1, 3) + oldText.substring(4);
// newText is "BCEF"; oldText is still "ABCDEF"

String s = "value=" + 2 + 3;       // "value=23"
String t = "value=" + (2 + 3);     // "value=5"
```

> Index 安全区
> 长度为 n 的 String，其合法 character index 是 0 到 n-1；substring 的 end index 可以等于 n，因为 end 不包含在结果中。

2.4 Math class

Method	用途
Math.abs(x)	absolute value
Math.pow(a, b)	a 的 b 次方，返回 double
Math.sqrt(x)	平方根，返回 double
Math.random()	返回 0.0 <= value < 1.0 的 double

Random range 模板

```
// Integer from min to max, inclusive
int value = (int) (Math.random() * (max - min + 1)) + min;

// Double from low (inclusive) to high (exclusive)
double value2 = Math.random() * (high - low) + low;
```

- inclusive integer 区间的数量是 max - min + 1。例如 25 到 60 一共有 36 个整数。
- cast 必须放在乘法之后；(int) Math.random() 永远是 0。
- 模拟两个骰子必须独立调用两次 Math.random()；把一次结果乘 2 不会得到正确分布。

2.5 Wrapper classes

Integer 与 Double 把 primitive value 包装为 object。Java 可自动进行 autoboxing 与 unboxing。

```
double a = 10.0;
Double b = 20.0;      // autoboxing
double c = b;         // unboxing
double total = a + b; // b is unboxed for arithmetic
```

Unit 2 高频题型

题型	快速策略
String tracing	给每个 character 标 index；按 left-to-right 计算 concatenation
method composition	先算最内层 method call，再把 return value 代入外层
random interval	先算区间 size，再 scale、cast、shift
valid call	核对 arguments -> parameters；核对 return value 的去向
overloading	比较 parameter types 与数量，不看 parameter 名称

Unit 2 自测清单

- 能否写出任意 inclusive integer random range？
- 能否解释 substring 的 end index 为什么不包含？
- 能否区分 void method 与 non-void method 的使用位置？
- 能否判断 String method 是否改变原 object？
- 能否用 parameter list 判断 overload 是否合法？

Unit 3 · Boolean Expressions and if Statements

本单元题量明显增加，重点不是背语法，而是证明 condition 的覆盖范围、分支关系与等价性。

3.1 Relational 与 logical operators

Operator	Meaning	Notes
== !=	equal / not equal	primitive 比 value；object reference 另见后文
< <= > >=	ordering comparison	注意 inclusive boundary
!	NOT	优先级高
&&	AND	两边都 true 才 true；short-circuit
||	OR	至少一边 true；short-circuit

Operator precedence（相关部分）

! -> arithmetic -> relational -> == / != -> && -> ||。考试中复杂条件建议主动加 parentheses。

3.2 if、if-else 与 else-if

```
if (score >= 90) {
    grade = "A";
} else if (score >= 80) {
    grade = "B";
} else {
    grade = "C or below";
}
```

- 多个独立 if：每个 condition 都会检查，可能执行多个 block。
- if / else-if / else chain：只执行第一个为 true 的 branch。
- 范围通常从最严格或最高 threshold 开始写，可避免重复上界。
- 没有 braces 时，if 或 else 只控制紧接着的一条 statement。
- else 总与最近一个尚未匹配的 if 配对（dangling else）。

> Boundary test
> 若分类规则含 999、1000、2999、3000 一类阈值，至少测试
> threshold-1、threshold、threshold+1。很多“看起来正确”的代码只在边界失败。

3.3 De Morgan's laws 与等价改写

Original	Equivalent
!(A && B)	!A || !B
!(A || B)	!A && !B
!(x < y)	x >= y
!(x == y)	x != y

口诀：否定进入括号时，AND / OR 互换，每个子条件也取反。

```
// Outside the inclusive range [0, 100]
value < 0 || value > 100

// Equivalent negation of being inside the range
!(value >= 0 && value <= 100)
```

3.4 Short-circuit evaluation

- A && B：若 A 已经 false，B 不会执行。
- A || B：若 A 已经 true，B 不会执行。

这不仅影响速度，也可保护危险操作。先检查 reference / index，再访问 object / String。

```
if (text != null && text.length() > 0) {
    System.out.println(text.substring(0, 1));
}
```

3.5 Comparing objects

Comparison	检查什么	String 建议
a == b	两个 reference 是否指向同一个 object	通常不要用于内容比较
a.equals(b)	按 class 定义比较内容 / logical equality	使用 equals

```
String a = new String("APCS");
String b = new String("APCS");
String c = a;

a == b        // false: different objects
a.equals(b)   // true: same contents
a == c        // true: same reference
```

> null 顺序
> 若 a 可能是 null，调用 a.equals(b) 会失败。可先判断 a != null；对已知 non-null literal，也可用 "AP".equals(a)。

3.6 判断两段代码是否 equivalent

不要凭感觉。把输入空间分区，再逐区比较。

Step	做法
1. 找 thresholds	列出所有比较中出现的临界值
2. 切分区间	例如 x < 0、x == 0、0 < x < 10、x == 10、x > 10
3. 追踪 branch	每个区间记录执行的 statement / return value
4. 找 counterexample	只要找到一个输入使输出不同，就不 equivalent

Unit 3 常见错误模式

- 把 if + if + else 误看成完整 chain；实际 else 只属于第二个 if。
- 高分段先赋值，后面的独立 if 又覆盖结果。
- 用 || 写“同时在两个边界之间”，导致条件几乎总为 true。
- 对 floating-point 运算结果直接用 ==；计算误差可能让数学等式不成立。
- 把 object 的 == 当作内容比较。

Unit 3 自测清单

- 能否用 De Morgan 改写任意 compound Boolean expression？
- 能否说明 short-circuit 如何避免 NullPointerException？
- 能否画出 else-if chain 的互斥区间？
- 能否解释 == 与 equals 对 String 的区别？
- 能否用一个 boundary counterexample 否定“equivalent”？

Unit 4 · Iteration

本次考试的 Unit 4 只复习 while loop 与 for loop：重点是执行顺序、循环次数、输出追踪，以及两种 loop 的等价改写。

4.1 while loop

```
initialization;
while (condition) {
    body;
    update;
}
```

每一轮顺序固定：检查 condition -> 执行 body -> update -> 回到 condition。若第一次 condition 为 false，body 执行 0 次。

常见 numeric patterns

```
// Count digits in a positive integer
int count = 0;
while (num > 0) {
    count++;
    num /= 10;
}

// Reverse the digits
int result = 0;
while (num > 0) {
    result = result * 10 + num % 10;
    num /= 10;
}
```

4.2 for loop

```
for (initialization; condition; update) {
    body;
}
```

任务	常用 header
0 到 n-1	for (int i = 0; i < n; i++)
1 到 n inclusive	for (int i = 1; i <= n; i++)
倒序 n-1 到 0	for (int i = n - 1; i >= 0; i--)
每隔 step	for (int i = start; i <= end; i += step)

> **while 与 for 等价条件**
> 必须同时保持相同 initialization、condition、body 与 update。尤其注意 update 在 print 之前还是之后。

while / for 高频检查

- 写出第一轮开始前的 initial value，以及每轮结束后的新值。
- 检查 condition 使用 < 还是 <=，确认最后一个 value 是否会进入 body。
- 确认 update 在每条可能执行的 branch 中都会发生，避免 infinite loop。
- 比较两个 loops 时，同时核对 initialization、condition、body、update 与 output position。

考前 Quick Sheet

主题	必须记住
integer division	int / int -> int；要小数就让运算前至少一个 operand 成为 double
casting	(int) 截断 toward zero；cast 太晚救不回已丢失的小数
digits	num % 10 取末位；num / 10 去末位
String index	0 到 length()-1；substring start inclusive, end exclusive
random int	(int)(Math.random() * (max-min+1)) + min
void	不能赋值、不能作为 println 的 value
De Morgan	!(A && B) == !A || !B；!(A || B) == !A && !B
String equality	内容用 equals；== 比 reference
if chain	独立 if 可执行多个；else-if chain 只执行第一个 true branch
loop order	condition -> body -> update -> condition

最后 5 分钟检查

- 每个 expression 的 type 是否正确？
- 所有 boundary 都是 inclusive 还是 exclusive？
- substring / loop 的最后一个 index 是否越界？
- else 属于哪个 if？
- loop variable 是否一定更新并最终结束？
- object 比较需要 == 还是 equals？
- 返回 double 时是否发生 integer division？

> **建议复习顺序**
> 先做 Unit 1 的计算准确性，再做 Unit 3 的逻辑边界，最后集中训练 Unit 4 的 while / for tracing 与等价改写。Unit 2 的 String 与 method 建议穿插复习。

Source basis: CSA WorkBook-1, Units 1-4. 内容按题型与知识点重新组织。